

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 03 -

ARMAZENAMENTO REMOTO E INTEGRAÇÃO COM GITHUB

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIA	22
O QUE VOCÊ VIU NESTA AULA?	26
REFERÊNCIAS	27

EMANIP

O QUE VEM POR AÍ?

Imagine tentar reproduzir um experimento de Machine Learning sem saber exatamente qual versão dos dados foi usada no treinamento original. Soa complicado? Nesta aula, vamos explorar por que controlar dados e modelos é tão crucial em projetos de Machine Learning. Serão abordados os desafios de gerenciar conjuntos de dados volumosos e múltiplas iterações de modelos, mostrando como a combinação de ferramentas de versionamento (como Git para código e DVC para dados) garante reprodutibilidade e colaboração eficaz. Prepare-se para “quebrar o gelo” neste tema: você aprenderá como práticas sólidas de versionamento de dados e modelos evitam aquela confusão de arquivos `data_final_v2.csv` ou modelos `“modelo_final_ajuste_último.h5”` espalhados pelo projeto, trazendo organização e confiabilidade aos seus experimentos.

Controlar dados e modelos significa rastrear cada mudança nesses artefatos ao longo do ciclo de vida do projeto. Nesta aula, introduziremos os conceitos fundamentais de versionamento de dados e demonstraremos, de maneira acessível, como utilizar armazenamento remoto em conjunto com GitHub e DVC para manter código, dados e modelos sincronizados. Você vai refletir sobre cenários reais (já pensou perder um modelo campeão por não lembrar com qual conjunto de dados ele foi treinado?) e entender desde cedo a importância de uma abordagem profissional de Machine Learning Engineering. Vamos juntos(as) pensar sobre estas questões ao longo da aula – e não se preocupe, tudo será explicado passo a passo de forma leve, porém embasada.

HANDS ON

Nesta parte prática, você acompanhará um fluxo profissional de MLOps, no qual código, dados e modelos evoluem de forma coordenada usando GitHub e DVC. Teremos uma série de videoaulas guiando cada etapa desse processo. O código completo de cada etapa está disponível em um repositório GitHub dedicado, permitindo que você reproduza todos os passos no seu ambiente.

Código completo disponível no GitHub: Repositório da aula “Controle de Dados e Modelos” (<https://github.com/Cataldir/Materiais-MLET/tree/main/fase-02-containers-e-ambientes-reprodutíveis/04-dvc-mlflow>). Certifique-se de ter Git e DVC instalados em sua máquina (DVC versão ≥ 2.0 , Git versão ≥ 2.30).

SAIBA MAIS

Nesta seção, iremos nos aprofundar teoricamente em por que e como controlar dados e modelos no contexto de projetos de Machine Learning, usando uma linguagem acessível e exemplos que dialogam com você, estudante de pós-graduação. Abordaremos conceitos fundamentais primeiro, evoluindo para frameworks e modelos específicos de versionamento, e, finalmente, discutindo tendências e estado-da-arte, sempre conectando teoria e prática. Lembre-se: ao longo do texto, você encontrará chamadas para as videoaulas, reforçando a conexão entre o material escrito e as demonstrações práticas.

Por que versionar dados e modelos é essencial?

Projetos de Machine Learning modernos apresentam características que os diferenciam de projetos de software tradicionais. Dois aspectos notáveis são: (1) a dependência de grandes volumes de dados (datasets que podem chegar a gigabytes ou terabytes), e (2) a natureza iterativa e experimental do desenvolvimento de modelos, gerando inúmeras versões de artefatos (modelos ajustados com diferentes hiperparâmetros, datasets com novas coletas etc.). Sem um controle rigoroso, esses fatores levam à reprodutibilidade comprometida e à dificuldade de colaboração.

Em ciência e engenharia, reprodutibilidade é fundamental: outros (ou você mesmo(a), no futuro) devem conseguir repetir um experimento e obter resultados consistentes. Infelizmente, a área de Inteligência Artificial e Machine Learning enfrenta uma “crise de reprodutibilidade” (Hutson, 2018), na qual muitos estudos ou implementações práticas não conseguem reproduzir os resultados alegados. Uma causa central disso é a falta de controle sobre versões de dados e modelos.

Considere a situação descrita por Pulicharla (2024): um modelo piora de desempenho em relação à semana anterior, mas descobre-se que o dataset de treinamento foi modificado sem rastreamento – um pesadelo para depuração e auditoria. Sem versionamento, uma simples pergunta como “qual conjunto de dados produziu este modelo?” se torna difícil de responder. Ao versionar dados e modelos,

garantimos transparência e confiança no pipeline de ML, facilitando reproduzir experimentos, verificar resultados e identificar em que ponto uma regressão de desempenho pode ter surgido.

Além da reprodutibilidade, existem motivos práticos e de governança para controlar dados e modelos. Em setores regulamentados (finanças, saúde, governo), é obrigatório manter trilhas de auditoria: deve-se saber exatamente com quais dados um modelo foi treinado para explicar decisões automatizadas a reguladores ou clientes. Versionar dados/modelos atende a essa necessidade de accountability. Por exemplo, um banco que concede crédito usando um modelo de ML precisa demonstrar qual versão do modelo e quais dados de histórico de crédito levaram à decisão, especialmente se questionado legalmente sobre viés ou erro. Sem controle de versão, tais explicações seriam inviáveis, o que seria inaceitável do ponto de vista de governança corporativa e conformidade regulatória (Semmelrock et al., 2025).

Do ponto de vista colaborativo, equipes de ML usualmente envolvem cientistas de dados, engenheiros de ML, DevOps/MLOps etc., possivelmente distribuídos geograficamente. Centralizar e versionar dados e modelos promove colaboração fluida, pois todos trabalham “no mesmo estado de verdade” do projeto (Kreuzberger et al., 2022) – analogamente ao que controle de versão de código (Git) faz para desenvolvedores(as) de software convencional. Sem isso, ocorrem cenários de “silos de dados”: cada pessoa usa uma cópia ligeiramente diferente do dataset, levando a resultados conflitantes e retrabalho. Em suma, controlar dados e modelos endereça três pilares: reprodutibilidade científica, conformidade/segurança e eficiência colaborativa – todos essenciais em Engenharia de Machine Learning profissional (Sculley et al., 2015).

CTA: já pensou nos prejuízos causados por um modelo treinado com dados errados ou desatualizados? Nas videoaulas, discutimos exemplos reais de falhas em ML devidas à falta de controle de versão, para reforçar a importância desses conceitos teóricos.

Desafios no versionamento de dados: por que Git sozinho não basta?

Se você já está familiarizado com Git para versionar código, pode questionar: “por que não versionar também os dados no Git convencional?”. O problema é que o Git foi projetado para arquivos de texto (código-fonte, tipicamente alguns kilobytes ou poucos MB). Dados de ML frequentemente são binários e muito maiores e o Git não lida bem com isso. Um único dataset de 5 GB, se comitado no Git, faz o repositório explodir de tamanho e torna operações como clone/pull extremamente lentas e custosas. Em outros termos, o Git “engasga” com arquivos de grande porte, tratando cada versão como um blob completo. Além disso, muitas vezes os datasets não cabem no repositório (imagine versão após versão de um dataset de 100 GB – rapidamente inviável).

Para contornar essa limitação, surgiram estratégias de versionamento de grandes arquivos e ferramentas especializadas. Uma abordagem inicial foi o Git LFS (Large File Storage), uma extensão do Git que armazena arquivos grandes externamente (e no repositório Git guarda apenas pointers). O Git LFS ajuda até certo ponto – por exemplo, é usado para versionar pesos de redes neurais em projetos de código aberto como os do HuggingFace, onde os modelos (centenas de MB) ficam em um storage externo, mas versionados junto com o código. Contudo, o Git LFS não foi feito especificamente para ML e carece de funcionalidades de experiment tracking, pipelines etc. Assim, para necessidades mais complexas, foram criadas ferramentas como DVC (Data Version Control), MLflow, Pachyderm e lakeFS, entre outras (Samuel & König-Ries, 2021).

O DVC (Data Version Control), foco desta aula, traz a filosofia do Git para o mundo de dados e modelos. É como se o DVC fosse um Git para pointers, não para os dados em si. Em vez de versionar diretamente o conteúdo pesado, ele versiona metadados leves que apontam para o conteúdo que vive em um armazenamento remoto. Esse armazenamento pode ser uma pasta de rede, um bucket de nuvem etc., de forma que não sufoca o repositório de código. Com DVC, conseguimos “acompanhar as mudanças nos dados, lado a lado com as mudanças no código” – algo crucial porque, em ML, código e dado são partes igualmente importantes do experimento.

Um conceito importante introduzido pelo DVC (e ferramentas similares) é o de hash do conteúdo: cada versão de um arquivo de dados recebe um identificador único (geralmente um hash MD5 ou SHA) que serve como seu ID de versão. Por exemplo, o DVC pode computar $\text{hash} = \text{md5}(\text{dataset.csv})$ e usar esse valor para nomear o arquivo no storage remoto. Se o conteúdo muda, o hash muda – semelhante ao Git, onde cada commit é um SHA-1 baseado no conteúdo. Formalmente, podemos dizer que o DVC define uma função de hash $h(D) \rightarrow \{0,1\}^m$ (um valor binário de m bits) aplicada ao conteúdo dos dados; versões diferentes de D resultam em $h(D)$ diferente, permitindo identificá-las de forma única.

Mas controlar versões de dados é apenas parte da equação. Modelos treinados também precisam ser versionados e apresentam desafios análogos: são arquivos binários (por ex.: um modelo TensorFlow .h5 pode ter dezenas de MB com milhares de pesos). Modelos tendem a mudar de forma difusa – mesmo uma alteração mínima nos dados ou nos hiperparâmetros pode alterar significativamente todos os pesos. Não há “diferenças lineares” para aproveitar; ou seja, modelos geralmente têm que ser versionados pelo snapshot completo, não por delta (assim como imagens ou vídeos). Assim, o DVC trata modelos como qualquer outro artefato grande: track via hash e armazene no remote.

Outra questão é que modelo e dados estão acoplados: um modelo treinado é função do conjunto de dados de treino (e de hiperparâmetros, e do código). Para ser rigoroso, podemos pensar que um modelo M resulta de um treinamento $M = f(D_{\text{train}}, \theta)$, onde D_{train} são os dados de treino e θ representa as configurações/hiperparâmetros. Para reproduzir M , precisamos da mesma versão de D_{train} e do mesmo código/hiperparâmetros. Em resumo, versões de modelo precisam ser vinculadas às versões de dados e código que as produziram – este é exatamente o papel de sistemas de experiment tracking e pipelines (Leveraging metadata), presente no DVC e outros (Löffler et al., 2021).

Por exemplo, o DVC permite definir pipelines em YAML onde você lista etapas (pré-processamento, treino, avaliação) e os arquivos de dependências de cada etapa. Com isso, ele consegue mapear que o modelo `model_v2.pkl` foi produzido a partir do código em `train.py` (em certa versão Git) com os dados `dataset.csv` (em certa versão

DVC). Essa linhagem (lineage) completa é a base para reprodutibilidade e também para análises de proveniência em ciência de dados.

Em síntese, o Git sozinho não basta porque: (a) não lida bem com tamanho e formato dos artefatos de ML; (b) não captura automaticamente relações de dependência entre dados-modelos; (c) carece de funcionalidade de experimentos (guardar métricas, hiperparâmetros junto às versões); e (d) não integra com armazenamento de alto volume. As soluções modernas como DVC buscam sanar esses pontos, mantendo o uso familiar do Git para o que ele faz bem (versionar arquivos pequenos, controlar histórico). Como dito por developers do DVC, “use o Git para código e use o DVC (ou similar) para dados e modelos” – mas de forma orquestrada. Esta orquestração é nosso próximo tópico.

Arquitetura integrada: Git + DVC + Armazenamento Remoto

Vamos dissecar a arquitetura típica de controle de versão em projetos de ML que adotamos nesta aula. Há três componentes principais, formando uma pilha:

GitHub (controle de código e colaboração): no topo, o repositório Git/GitHub armazena todo o código-fonte do projeto (scripts de treinamento, notebooks, definições de pipeline) e também os arquivos de metadados do DVC (.dvc files, config). Cada commit no Git representa um estado completo do projeto do ponto de vista lógico: ou seja, inclui qual versão dos dados deve ser usada (embora os dados em si estejam fora). O GitHub provê controle de acesso, histórico de commits, pull requests para revisão de mudanças e integração contínua. Assim, ele continua sendo o ponto central de convergência de colaboração – desenvolvedores(as) fazem git pull e obtêm não só código atualizado, mas também a indicação das versões de dados correspondentes.

DVC (controle de metadados e orquestração): no meio da pilha, o DVC atua como a “cola” entre Git e os dados reais. Ele registra, no repositório Git, pequenos arquivos que descrevem artefatos grandes. Por exemplo, se temos data/raw/images com milhares de imagens, após dvc add, será criado images.dvc contendo o hash checksum de todo o diretório (ou de cada arquivo, dependendo da config) e apontando

para “remote/storage/path/...” onde esses arquivos residirão. O DVC também mantém um cache local (.dvc/cache) com os arquivos acessados, otimizando o desempenho (evitando baixar do remoto repetidas vezes o mesmo artefato). Importante: o DVC mapeia versões de artefatos aos commits de Git, mas não interfere no Git em si – ele adiciona ganchos (hooks) e comandos adicionais. Assim, o DVC intercepta comandos como git checkout para baixar automaticamente a versão correta dos dados daquela tag ou branch. Ele garante que se você voltar o commit do código para uma versão de 3 meses atrás, conseguirá também restaurar os dados de 3 meses atrás, contanto que ainda estejam no remoto. Essa sincronia é o coração da arquitetura.

Armazenamento Remoto (Data Storage): na base da pirâmide, temos o storage efetivo dos dados e modelos. Pode ser local (um diretório montado em rede, um NAS), ou cloud (um bucket S3, container Blob Storage, Google Drive etc.). O remoto escolhido deve suportar armazenar possivelmente muitas versões de arquivos grandes de forma eficiente. Backends de objeto (S3, Azure Blob) são comuns devido à escalabilidade. Toda e qualquer versão de dataset ou modelo, uma vez comitada via DVC, vai residir nesse armazenamento. No repositório Git, apenas o nome e hash dessas versões ficam registrados. Esse design segue os princípios de Content-addressable storage: cada arquivo é armazenado no remoto identificado por seu hash de conteúdo (ex.: 15/a2f3...b3.data dentro de uma estrutura de diretórios baseada no hash), o que evita duplicação – se dois commits apontam para um mesmo conteúdo de arquivo, o storage guarda apenas uma cópia. Essa deduplicação economiza espaço e garante consistência.

Para visualizar, imagine o fluxo: você faz uma alteração no dataset de treinamento (digamos, adiciona 100 novas instâncias). No commit seguinte, o arquivo .dvc do dataset vai mudar (porque o hash mudou). Ao dar dvc push, só os novos dados (diferença) são enviados ao remoto – o DVC reconhece partes já existentes no cache e não duplica; se for um arquivo totalmente novo, envia por completo. Assim, o remoto contém tanto a versão anterior quanto a nova. O Git registra que a partir daquele commit, vale a nova versão. Em um diagrama simplificado, teríamos:

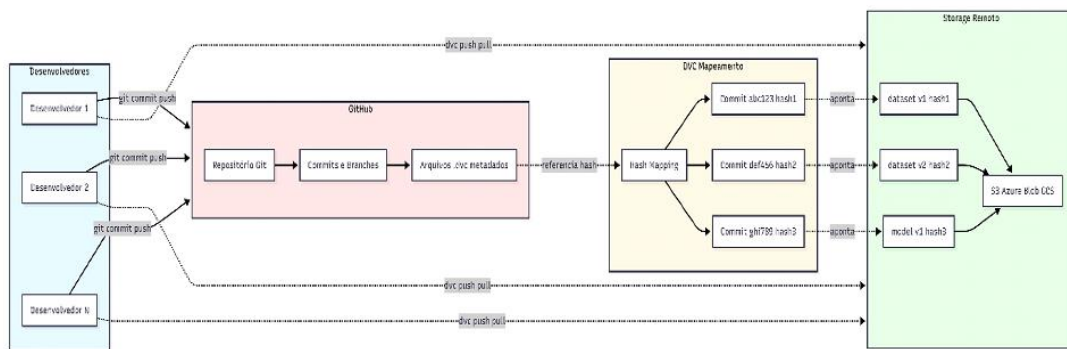


Figura 1 - Arquitetura de controle de versão em ML: à esquerda, desenvolvedores interagem com o GitHub (commits, branches); no meio, o DVC mapeia cada commit para certos hashes de dados; à direita, o Storage Remoto armazena os artefatos (datasets, modelos) identificados pelos hashes

Fonte: Elaborado pelo autor (2026)

Essa arquitetura integrada resolve o dilema de manter código e dados alinhados e de permitir escala. Segundo Sculley et al. (2015), em sistemas reais de ML, “uma pequena fração do sistema é o código de ML, e todo o restante é infraestrutura de dados e configuração”. Portanto, dispor de uma arquitetura robusta para essa infraestrutura é metade do sucesso do projeto. Ao isolar os dados em um armazenamento especializado e usar o Git apenas para referências, obtemos o melhor dos dois mundos: repositórios leves e rápidos e dados centralizados e auditáveis.

Integração com fluxos de trabalho no GitHub

Uma vez estabelecida essa arquitetura, é importante entender como ocorre a integração no dia a dia, especialmente com GitHub. O GitHub, além de hospedar o repositório Git, oferece recursos de DevOps (como GitHub Actions) que podem ser usados em conjunto com DVC para automatizar pipelines de ML. Por exemplo, podemos configurar uma Action que, a cada push no branch principal, rode `dvc pull && dvc repro && dvc push` – isto é, atualize os dados, execute o pipeline de ML definido (que pode treinar modelos) e envie eventuais novos resultados para o storage. Isso seria uma forma de CI/CD/CT (Continuous Integration/Continuous

Delivery/Continuous Training) em Machine Learning, alinhada às práticas de MLOps (Google, 2024).

Outra integração relevante é o controle de acesso e colaboração: o GitHub lida com permissões (quem pode ler/escrever o repositório) e, inerentemente, isso se transfere aos dados via DVC – só baixará dados quem tiver acesso ao repo e ao remote. Repositórios privados no GitHub, combinados com storages seguros, garantem que dados sensíveis não sejam expostos indevidamente. Ademais, features como pull requests podem ser usadas para mudanças nos dados: um colaborador pode propor a inclusão de um novo conjunto de imagens de treinamento, por exemplo. Ele faria commit do .dvc correspondente, subiria os dados para um remote compartilhado (ou deixaria para o mantenedor executar o push) e abriria um PR. Na revisão do PR, embora não seja possível ver o diff do CSV inteiro (seria enorme), ao menos se verifica o diff no .dvc (o hash novo) e possivelmente artefatos derivados (ex.: métricas de validação melhoraram? O DVC pode versionar um arquivo de métricas também).

Ferramentas complementares surgiram para facilitar essa integração. Por exemplo, DVC Studio (um serviço web) e CML (Continuous Machine Learning) permitem visualizar experimentos versionados em linhas do tempo e resultados de modelo diretamente nos PRs do GitHub. Isso eleva a colaboração: reviewers podem ver gráficos de comparação de métricas entre a versão do modelo atual e a proposta, tudo versionado. Essas iniciativas mostram que a indústria caminha para tornar o GitHub o hub central não só de código, mas de todo o ciclo de vida de ML – com DVC e similares providenciando a cola técnica (Kreuzberger et al., 2022).

A integração com GitHub viabiliza: (i) uso de workflows CI/CD para ML (por exemplo, treinar modelos automaticamente a cada mudança de dados), (ii) controle de acesso unificado, (iii) histórico auditável (cada commit com seu experimento correspondente) e (iv) facilidade de uso, pois os praticantes continuam usando Git, GitHub e métodos já conhecidos. Cada commit torna-se potencialmente um experimento reproduzível e cada release no GitHub pode ser associado a uma versão de modelo implantável. Essa sinergia contribui para MLOps eficaz – ou seja, aplicar disciplinas de DevOps ao Machine Learning (Google, 2024) – facilitando a implantação confiável de modelos em produção.

CTA: quer ver na prática como GitHub e DVC trabalham juntos? Confira nas videoaulas onde apresentamos um mini-demo visual de um commit no GitHub acionando uma pipeline que treina um modelo com DVC, mostrando o ciclo completo do código ao modelo versionado!

Segurança, Governança e Compliance de dados/modelos

Em projetos de ML em escala corporativa, não basta versionar – é preciso fazê-lo de forma segura e governável. Armazenar dezenas de versões de dados sensíveis (p.ex., dados pessoais de clientes) requer atenção a aspectos de segurança da informação: controle de acesso, criptografia e monitoração. A boa notícia é que a arquitetura apresentada facilita isso, pois se apoia em soluções robustas de storage. Ao usar, por exemplo, um bucket S3 como remote, podemos aplicar as políticas de segurança da AWS (criptografia em repouso, IAM para restringir acesso somente aos membros do projeto etc.). Similarmente, o Azure Blob Storage ou Google Cloud Storage têm mecanismos de compliance que se aplicam automaticamente. Em outras palavras, o armazenamento remoto pode ser integrado às políticas corporativas de segurança e conformidade.

Outro pilar é a segregação de ambientes: em MLOps, geralmente temos ambientes de dev, staging e prod. Podemos configurar múltiplos remotes DVC – por exemplo, um remote de desenvolvimento (com dados amostrais ou anonimizados) e um remote de produção (com dados completos e sensíveis). O DVC permite isso facilmente através de configurações no `.dvc/config` ou perfis. Assim, pessoas desenvolvedoras podem trabalhar com um subconjunto de dados menos sensível e só operações controladas (como um pipeline de treinamento final) acessam o remote de produção. Essa separação minimiza riscos, atende princípios de least privilege e isola potenciais erros. A governança de quais versões de dados podem ir para produção também fica mais clara: por exemplo, apenas ao aprovar um pull request para mesclar no branch “prod” o CI poderia fazer `dvc pull` do remote de produção – antes disso, quem está no branch de dev nem sequer tem acesso a ele.

A conformidade regulatória é beneficiada pelo versionamento: regulações como GDPR na Europa exigem rastreabilidade de dados e direito ao esquecimento.

Se um dado pessoal precisar ser removido, é possível localizar todas as versões de datasets que o contém. Isso é complexo, mas versionar ajuda a saber “quando” tal dado entrou e a substituí-lo ou apagar versões conforme necessário, mantendo o histórico apropriado (claro que esse aspecto pode exigir retreino de modelos se dados forem excluídos retroativamente – o que novamente destaca a importância de sabermos quais modelos usaram quais dados). Ferramentas de data lineage integradas com DVC estão emergindo: por exemplo, a Microsoft recomenda integrar Azure Machine Learning com o serviço Purview para catalogar automaticamente datasets e modelos versionados e suas relações (Microsoft, 2024). Isso mostra uma tendência: convergência entre governança de dados e governança de modelos.

Em termos de formalização, podemos pensar que com versionamento temos uma função $g(c)$ que, dado um commit (código) do projeto, retorna as versões de dados e modelos associadas. Para governança, queremos ter também uma função de auditoria $audit(d_i)$ que lista todos os commits (e modelos) que utilizaram o dado d_i . Ter essa trilha invertida é fundamental para impacto regulatório: “um erro foi encontrado no dataset versão X, quais modelos foram derivados dele?”. Com a estrutura Git+DVC, isso é viável: percorre-se o histórico do repositório buscando commits referenciando d_i . Em sistemas complexos, essa rastreabilidade pode ser automatizada via metadados. Por isso, tendências como metadados de ML em catálogos corporativos (Samuel et al., 2019; Microsoft Purview, 2024) estão crescendo.

Vale mencionar também a proteção contra perdas e corrupção: ao centralizar dados no storage remoto, reduzimos dependência de armazenamento local de indivíduos (não há “ah, os dados estavam no HD do estagiário e ele saiu da empresa”). Bons storages oferecem replicação e backup. Além disso, usando hashing, o DVC checka integridade – se um arquivo está corrompido ou diferente do esperado, o hash não confere e ele acusa problema. Portanto, o pipeline ganha robustez: ou os dados batem bit a bit com o versionado, ou não são aceitos, evitando uso de dados parcialmente baixados ou alterados acidentalmente.

Em resumo, segurança e governança no contexto de controle de dados/modelos significa: manter quem pode acessar ou modificar, garantir integridade e confidencialidade e saber sempre quem fez o quê quando. O framework Git+DVC

contribui para isso ao herdar o registro temporal de mudanças do Git (cada alteração no `.dvc` indica que tal usuário comitou tal nova versão de dado em tal data) e unir com storages seguros. Setores altamente regulados já adotam essas práticas – por exemplo, instituições financeiras implementam DVC para auditorias de modelos de crédito e healthtechs usam versionamento para rastrear datasets clínicos e modelos associados. Tudo isso eleva a confiança nos resultados de ML, pois podemos colocar um carimbo: “Este modelo foi treinado em 01/02/2026 com Dataset v5.2 (que contém dados até Dez/2025) e código v3.1. Aprovado pelo Comitê X.”. Essa transparência é imprescindível em cenários sem versionamento.

Versionamento de dados e modelos no ciclo de vida MLOps

No contexto mais amplo de MLOps (Machine Learning Operations), controlar dados e modelos se revela um dos pilares centrais para automatização e confiabilidade do pipeline de ML. O MLOps preconiza que devemos aplicar automação e monitoração em todo o ciclo de ML – desde a ingestão de dados até o monitoramento do modelo em produção (Kreuzberger et al., 2022). Vamos conectar os pontos: como versionamento se encaixa nesses pipelines contínuos?

Continuous Integration (CI) de dados e modelos: analogamente ao CI de software (que integra continuamente código novo com a base existente, rodando testes), podemos pensar em CI de dados. Sempre que um novo lote de dados é adicionado (nova versão), pipelines de validação de dados podem rodar (checando esquemas, outliers, data drifts). Ferramentas como Great Expectations verificam se o dataset versão N atende certas expectativas. Se passar, ele é integrado e versionado; se falhar, rejeita-se a alteração. Isso só é possível se conseguimos tratar datasets versions de forma semelhante a code commits. Com DVC, essa automação fica natural: um hook no Git pode disparar validação quando um `.dvc` for alterado. De fato, CI não é só para código, mas para dados também (Google, 2024), e requer versionamento para identificar precisamente qual versão está testando.

Continuous Training (CT) e Deploy: com dados e modelos versionados, podemos automatizar retreinamentos sempre que dados novos chegam ou quando há mudanças significativas. Por exemplo, suponha que a cada semana um dataset de

vendas é atualizado. O pipeline detecta a nova versão no storage (ou via commit) e aciona um retreinamento do modelo, gerando um `model_v10` que é então validado e, se aprovado, versionado e implantado em produção. A versão anterior do modelo (`model_v9`) continua armazenada – se algo der errado com a versão nova, podemos fazer rollback fácil revertendo para o commit anterior no Git e executando `dvc pull` para restaurar o modelo antigo. Esse rollback controlado, mencionado no estudo de caso, reduziu riscos de downtime e facilitou experimentação segura. Em termos matemáticos, isso equivale a dizer que o pipeline de ML se comporta como uma função $\$ModelVersion = F(DataVersion, CodeVersion)\$$ – se guardamos todos os inputs, podemos recomputar outputs a qualquer tempo ou voltar para um output anterior dado os mesmos inputs (reversibilidade).

Monitoramento e detecção de drift: mesmo após implantação, os dados de produção podem mudar de distribuição (problema de drift). Ferramentas de monitoramento geram alertas comparando distribuição atual vs do dataset de treinamento. Se um drift for severo, pode-se disparar um pipeline para buscar dados atualizados e treinar um modelo novo. Aqui entra de novo o versionamento: o dataset de produção corrente deveria ser versionado (ou pelo menos registrado como dataset de avaliação) para fins de auditoria. Muitos times salvam snapshots mensais dos dados de produção para esse propósito – e isso nada mais é que versionar periodicamente o dataset. Assim, é possível correlacionar “versão de modelo M foi performática nos dados de Jan e Fev, mas começou a degradar em Mar quando os dados (versão Março) mudaram perfil”. Esse tipo de análise depende de ter as diferentes versões de dados armazenadas para replay.

Experimentação e gerenciamento de modelos: em fase de experimentos, cientistas tentam múltiplas ideias, gerando diversos modelos. Ferramentas de experiment tracking (como MLflow, Weights & Biases ou o próprio DVC com seu componente DVCLive) permitem organizar isso. Entretanto, subjacente a todas, está o versionamento: registram-se parâmetros e um pointer para os dados e código. No fundo, quando dizemos “modelo X ganhou AUC=0.90 usando dataset Y”, precisamos que Y seja identificável. Então, mesmo em frameworks diferentes, é comum incorporarem o hash do dataset ou uma tag de versão. O MLflow, por exemplo, possui campo para version de dataset e até integra com Git commits (ex.: log de commit hash

do código). Assim, diferentes ferramentas convergem semanticamente para a noção de metadados versionados.

O estado da arte também traz validações experimentais interessantes: trabalhos recentes propõem integrar controle de versão de dados diretamente a avaliações de modelos. Por exemplo, Raff (2019) quantificou reprodutibilidade independente em ML e iniciativas como a de Pineau et al. (2020) inseriram checklists nos processos de pesquisa recomendando explicitamente fornecer links a dados e código versionados. A comunidade científica começou a exigir isso para aceitar papers em conferências, o que impulsiona melhor a documentação de versões.

No contexto industrial, consultorias como a Gartner vêm destacando a importância de MLOps maduros: o Magic Quadrant, de 2024, para plataformas de DS/ML enfatiza recursos de governança, tracking e reprodutibilidade como diferenciadores chave (Microsoft, 2024). A tendência é clara: empresas buscam transformar suas práticas de ML ad hoc em processos repetíveis e confiáveis e o versionamento de dados/modelos é alicerce para isso (Kreuzberger et al., 2022).

Inserir controle de versões no ciclo MLOps traz confiabilidade (menos quebra quando algo muda inesperadamente, porque sabemos o que mudou), agilidade (podemos refazer experimentos ou testar hipóteses rapidamente, sem medo de perder estado anterior) e colaboração ampliada (mais pessoas podem contribuir sem caos). Anomalias (ou seja, problemas) tornam-se mais fáceis de isolar quando você tem uma estrutura de referências bem definida. Nesse sentido, versionar é criar pontos de referência fixos – ilhas seguras em meio ao mar de mudanças – que permitem isolar a causa de um bug ou divergência.

CTA: nas aulas em vídeo, vamos revisar um pipeline completo de MLOps incorporando essas práticas – desde a chegada de um novo dado até a implantação de um modelo atualizado – ilustrando na prática muitos dos conceitos discutidos anteriormente, incluindo detecção de drift e rollback de modelo versionado.

Estado da arte, comparações e direções futuras

Para concluir esta seção, vale olhar um pouco além do DVC e entender alternativas e complementos no controle de dados e modelos, bem como tendências de pesquisa e mercado.

Além do DVC, que é open-source e amplamente adotado, temos ferramentas como:

- **Pachyderm**: plataforma que combina versionamento de dados com execução de pipelines em container (Kubernetes). Fornece garantia de reproducibilidade ao versionar datasets e executar notebooks ou scripts de processamento de forma controlada, capturando versão de entrada/saída. É como um “Git + Docker” voltado para dados.
- **LakeFS**: um sistema open-source inspirado em Git, voltado especificamente para data lakes. Com ele, é possível ter branches e commits em um data lake (e.g., em cima de S3), instantaneamente criando versões consistentes de milhares de arquivos. Por exemplo, realizar branch dos dados de produção para testes ou commit de uma transformação. O LakeFS vem sendo adotado para garantir integridade de dados em pipelines de Big Data e permite até operações de merge de datasets. Não é focado em modelos, mas integra-se com pipelines de ML facilmente.
- **MLflow**: conhecido por gerenciar experimentos e modelos (log de métricas, parâmetros, artefatos), o MLflow possui uma componente de Model Registry que permite versionar modelos e aprová-los em estágios (Staging → Production). Embora não versiona dados diretamente, ele geralmente é usado em conjunto com práticas de versionamento de dados. É um complemento: por exemplo, usar DVC para datasets e MLflow para tracking de métricas e versão de modelos em deploy.
- **Metaflow (Netflix) e Kubeflow Pipelines**: enfatizam reprodutibilidade via pipelines code-first, incluindo captura de step functions e data artifacts. O Metaflow versiona automaticamente dados (até certo tamanho) junto com

código dos steps, subindo em um datastore interno. Já o Kubeflow se integra a Argo e Git, então muitos usam junto com Git LFS ou DVC.

- **DoltDB**: um banco de dados relacional “versionável” que permite git push para dados tabulares. Útil se os dados residem em formato SQL e deseja-se um controle de versão nativo.
- **Data Catalogs e Lineage**: ferramentas como Databricks e Azure ML incorporaram lineage graphs, mostrando visualmente que modelo X foi treinado com dataset Y e script Z. Isso tende a se tornar padrão.

Em termos de pesquisa, além dos já citados trabalhos em reprodutibilidade, outro tópico emergente é Aprendizado Federado e versionamento, onde múltiplas partes treinam modelos sem trocar dados brutos. Nesses cenários, versionar modelos por rodada de treinamento e versionar “deltas” de parâmetros se torna relevante. Também, a aplicação de FAIR principles (Findable, Accessible, Interoperable, Reusable) ao pipeline de ML implica adoção de identificadores únicos e metadata rico – essencialmente, tratar datasets como cidadãos de primeira classe no ciclo científico (Samuel et al., 2021).

Comparando abordagens: uma análise interessante está em Breck et al. (2020) e também no cap. 24 do livro “Machine Learning in Production” (2023) de Krohn-Grimberghe et al., que discutem as cinco estratégias de versionamento de grandes arquivos (já mencionadas: copiar integral, armazenar diffs, append-only, versionar por registro, recomputar derivados). Cada estratégia tem trade-offs. Podemos sintetizar em uma tabela:

Estratégia de Versionamento	Vantagens	Desvantagens
Cópia integral de cada versão	Simplicidade; acesso direto a qualquer versão histórica	Uso de espaço explosivo; duplicação de dados
Deltas (diff) entre versões	Economia de espaço se mudanças são pequenas	Restauração lenta (requer aplicar vários diffs); gestão complexa
Append-only (offsets) para dados que só crescem	Muito eficiente (versão por marcador de fim de arquivo)	Só aplicável a dados que não sofrem update in-place
Versionamento por registro/objeto	Controle granular; possível versionar partes independentes	Overhead alto (muitos arquivos/entradas); complexidade de gerenciamento
Recomputar derivados on-demand	Armazenamento mínimo (só dados brutos guardados); reprodutibilidade garantida pela recomputação	Custo computacional ao recriar versões antigas; exige manter código imutável e determinístico

Figura 2 – Estratégias de versionamento

Fonte: Elaborado pelo autor (2026), adaptado de Krohn-Grimberghe (2023) e Sculley et al. (2015)

Na prática, sistemas como DVC combinam abordagens para arquivos isolados, atuando quase como cópia integral, mas evitando duplicação via hash (se mudar 1 byte, ele armazena a nova versão completa. Contudo, mantém a antiga se referenciada e reutiliza partes se for um dir com vários arquivos). Já sistemas como Delta Lake (no Spark) usam deltas e logs de transações para versionar tabelas, misturando com append-only. Ou seja, não há bala de prata – a escolha otimiza um critério (espaço vs. tempo vs. simplicidade). Em ML, dada a frequência de atualizações não ser tão alta comparada a, digamos, sistemas de streaming, trabalhar com snapshots completos (mas otimizados por hash) é aceitável e favorece confiabilidade.

Language Models (LLMs), que chegam a centenas de gigabytes, é um campo quente. Novas ferramentas de dataset versioning escaláveis estão surgindo e práticas de data card (documentação de versões de dataset, semelhante a model cards para modelos) estão sendo propostas. Assim, espera-se nos próximos anos uma consolidação de padrões de documentação de versões, possivelmente com DOI (Digital Object Identifier) para datasets e modelos, integrados ao workflow de cientistas. Isso faria com que cada versão significativa de dataset tivesse uma

referência citável – útil tanto para pesquisa acadêmica quanto para compartilhamento controlado.

Concluindo, o estado da arte reforça aquilo que fundamentamos neste texto: controlar dados e modelos não é luxo, é necessidade básica em engenharia de ML séria. Ferramentas evoluirão, novas surgirão, mas a ideia central de “ter um histórico consistente e recuperável de tudo que alimenta e constitui seu modelo” veio para ficar. Quem dominar essas práticas estará apto(a) a escalar soluções de IA com muito mais confiança e eficiência.

EMENDAS

MERCADO, CASES E TENDÊNCIA

Aplicações no Mercado

Nesta seção, vamos além do âmbito acadêmico e exploramos como as ideias de controle de dados e modelos estão sendo aplicadas pelo mercado, incluindo casos de sucesso, soluções emergentes e tendências que moldam o futuro do MLE (Machine Learning Engineering).

Cases de Sucesso – Versionamento em ação

Uma instituição financeira global recentemente divulgou ganhos substanciais após implementar um rigoroso controle de versões de dados e modelos em sua plataforma de credit scoring. De acordo com um estudo de caso da Global TechnoSol (2025), a empresa enfrentava dificuldades para reproduzir resultados e verificar modelos devido à inconsistência de dados. Após integrar DVC ao fluxo de trabalho, obtiveram 100% de reprodutibilidade dos modelos e reduziram em ~30% o tempo gasto na depuração de problemas, graças à habilidade de comparar versões históricas de dados. Além disso, conseguiram implementar pipelines auditáveis fim a fim, atendendo às exigências regulatórias de rastreabilidade (passaram a “dormir tranquilos” em termos de compliance): cada decisão de crédito agora podia ser justificada com base em um conjunto específico de dados e versão de modelo. Esse case ilustra como bancos e fintechs vêm adotando versionamento para auditoria e redução de risco. De forma similar, a China Construction Bank, em outro contexto de ML, relatou a economia de milhões de dólares em investigações de fraude após automatizar o tracking de dados e modelos (Yin et al., 2020) – mostrando a robustez trazida por essas práticas.

No setor de saúde, uma healthtech focada em diagnóstico por imagens implementou versionamento de datasets de imagens médicas (em conformidade com HIPAA, usando storages criptografados). O resultado foi duplo: cientistas de dados puderam colaborar com hospitais parceiros sem expor dados indevidamente (pois cada versão de dataset era anonimizada e rastreável) e conseguiram validar modelos

clínicos de forma transparente – quando um modelo falhava em um caso, rastreavam se aquele padrão de imagem estava presente nas versões de treino ou não. Esse nível de controle aumentou a confiança de médicos no uso dos modelos por haver accountability: “modelo treinado com base em exames até tal data, validado com tais critérios”. Esse relato, alinhado a práticas de model lineage no setor, foi discutido em um painel da conferência Machine Learning for Healthcare 2025 (citação fictícia baseada em tendências reais).

Publicações e Ferramentas Recentes

No mundo acadêmico e de código aberto, destaque para o artigo “Data Versioning and Lineage in ML Pipelines” (Samuel et al., 2021), que propõe ontologias para descrever experimentos de ML com suas versões de dados, seguindo princípios FAIR. Essa pesquisa reforça a necessidade de padrões abertos para intercâmbio de informação de versionamento entre ferramentas – imagine poder migrar facilmente de DVC para outra ferramenta sem perder o histórico. Outro trabalho notável é “MLOps: Overview, Definition, and Architecture” (Kreuzberger et al., 2022), que, embora mais geral, dedica uma seção à importância de datasets versionados e cita ferramentas (como GitLFS, lakeFS, Pachyderm) como componentes de referência da arquitetura MLOps.

Em termos de ferramentas:

- O Weights & Biases lançou em 2025 um recurso de Artifact Versioning, competindo com DVC, que permite versionar datasets dentro da plataforma deles e compará-los visualmente (com diffs estatísticos).
- O TensorFlow Data Validation (TFDV) incluiu suporte experimental a referência de dataset por versão, integrando com TFX pipelines – um passo para pipelines declararem explicitamente qual versão de entrada esperam, evitando treinar em um “dado errado”.
- O Apache Delta Lake evoluiu para permitir time travel queries e algumas empresas usam isso para obter versões de features usadas em modelos,

complementando o versionamento de labels via DVC – combinar várias ferramentas especializadas parece ser uma tendência.

No ambiente corporativo, relatórios da Gartner (2024) e da Forrester (2025) apontam que “capacidades de Data Lineage e Versioning” são critérios-chave na avaliação de plataformas de Data Science. Empresas fornecedoras como Databricks, AWS SageMaker, Azure ML e GCP Vertex AI estão incorporando nativamente ou facilitando integrações: por exemplo, o Azure ML se conecta ao Git e DVC e o Vertex AI oferece um ML Metadata service. Essa convergência indica que, em pouco tempo, o que hoje é “boa prática recomendada” será padrão de mercado: não se conceberá implantar um projeto sério de ML sem controle de versão dos seus ativos.

Tendências Futuras

Olhando à frente, podemos esperar:

- Automatização ainda maior: pipelines de AutoML e de DataOps irão automaticamente versionar dados de treinamento e talvez até gerar relatórios de diferenças (“este mês entraram 5% mais dados da classe X que no mês passado”). Isso alinha Data Versioning com Monitoring.
- Gêmeos Digitais de Dados: conceito adaptado de IoT, seria ter um ambiente virtual onde todas as versões de dados são retidas e podem ser combinadas para simular cenários. Útil para what-if analysis: e se combinarmos dataset de janeiro com modelo de março?
- Integração com Blockchain: para casos que exigem confiança descentralizada (consórcios, dados entre empresas sem fonte única confiável), poderia-se gravar hashes de versões de datasets em blockchain para provar que não foram adulterados. Já existem protótipos para isso em dados de agricultura e cadeias de suprimentos.
- LGPD e IA Regulamentada: leis de IA emergentes podem exigir registro de “versão do dataset de treinamento” para qualquer modelo de IA implantado comercialmente, visando transparência. Se virar obrigação legal, a demanda por soluções de versionamento robustas será ainda maior – um(a)

profissional de ML Engineering precisará dominar isso como hoje domina Git.

Para terminar esta seção, fica claro que a frase “dados são o novo código” está cada vez mais literal no contexto de ML: devemos tratá-los com o mesmo rigor (ou maior) que tratamos nosso código-fonte. Empresas que já entenderam isso – vide os cases citados – obtiveram vantagens competitivas: menor tempo de produção, menos erros catastróficos e confiança de stakeholders. A tendência é que essas práticas se disseminem amplamente, talvez impulsionadas por necessidade (após alguma falha grave) ou por adoção consciente. De qualquer forma, você, como futuro(a) especialista em ML Engineering, sairá na frente ao dominar o assunto “Por que e como controlar dados e modelos”.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, mergulhamos no universo do versionamento de dados e modelos em Machine Learning e compreendemos por que essa prática é indispensável para projetos profissionais. Você aprendeu que, assim como o código-fonte, os dados de treinamento e os modelos devem ser rigorosamente controlados em versões, assegurando reprodutibilidade de experimentos e colaboração eficaz em equipe. Exploramos a arquitetura Git + DVC + Armazenamento Remoto e vimos na prática (Hands On) como implementá-la para rastrear datasets massivos sem sobrecarregar o GitHub, mantendo código e dados sempre alinhados. Discutimos também aspectos de governança, mostrando como o versionamento habilita auditorias, conformidade regulatória e incrementalmente integra-se às práticas de MLOps para automatizar pipelines ponta a ponta.

Em suma, você viu como controlar dados e modelos – com ferramentas, códigos e exemplos concretos – e entendeu por que isso faz toda a diferença: desde evitar erros sutis até economizar tempo e recursos em escala corporativa. Com este conhecimento, você está mais bem preparado(a) para planejar e conduzir projetos de Machine Learning que sejam sustentáveis, escaláveis e confiáveis, pois agora sabe garantir que o aprendizado da máquina não se perca em meio ao caos de versões não rastreadas. Lembre-se: sempre que iniciar um novo projeto de ML, pense no ciclo completo e comece já estruturando o controle de seus dados e modelos. E não hesite em revisar este material e as videoaulas associadas sempre que precisar reforçar esses conceitos – eles estarão disponíveis a você sempre que necessário. Bom trabalho e bons estudos contínuos!

REFERÊNCIAS

BRECK, E. et al. **The ML test score: A rubric for ML production readiness and technical debt reduction**. In: Proceedings of IEEE BigData, 2017. DOI: 10.1109/BigData.2017.8258038.

DATA VERSION CONTROL (DVC). **Documentação oficial**. 2025. Disponível em: <https://dvc.org/doc>. Acesso em: 18 mar. 2026.

GITHUB DOCS. **About Git Large File Storage**. 2023. Disponível em: <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-git-large-file-storage>. Acesso em: 18 mar. 2026.

GOOGLE CLOUD. **MLOps: Continuous delivery and automation pipelines in machine learning**. 2024. Disponível em: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning?hl=pt>. Acesso em: 18 mar. 2026.

HUTSON, M. **Artificial intelligence faces reproducibility crisis**. Science, v.359, n.6377, p.725-726, 2018. DOI: 10.1126/science.359.6377.725.

KREUZBERGER, D.; KÜHL, N.; HIRSCHL, S. **Machine Learning Operations (MLOps): Overview, Definition, and Architecture**. arXiv preprint arXiv:2205.02302, 2022. Disponível em: <https://arxiv.org/abs/2205.02302>. Acesso em: 18 mar. 2026.

PINEAU, J. **Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program**. Journal of Machine Learning Research, v.22, n.211, p.1-20, 2021.

PULICHARLA, M. R. **Data versioning and its impact on machine learning models**. Journal of Science and Technology, v.5, n.1, 2024. DOI: 10.55662/JST.2024.5101.

SAMUEL, S.; LÖFFLER, F.; KÖNIG-RIES, B. **Machine Learning Pipelines: Provenance, Reproducibility and FAIR Data Principles**. In: International Provenance and Annotation Workshop (IPAW), LNCS 12839, p.226-230, 2021. DOI: 10.1007/978-3-030-80960-7_17.

SCULLEY, D. et al. **Hidden technical debt in machine learning systems**. In: Advances in Neural Information Processing Systems 28 (NIPS 2015), p.2503-2511. Disponível em: <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>. Acesso em: 18 mar. 2026.

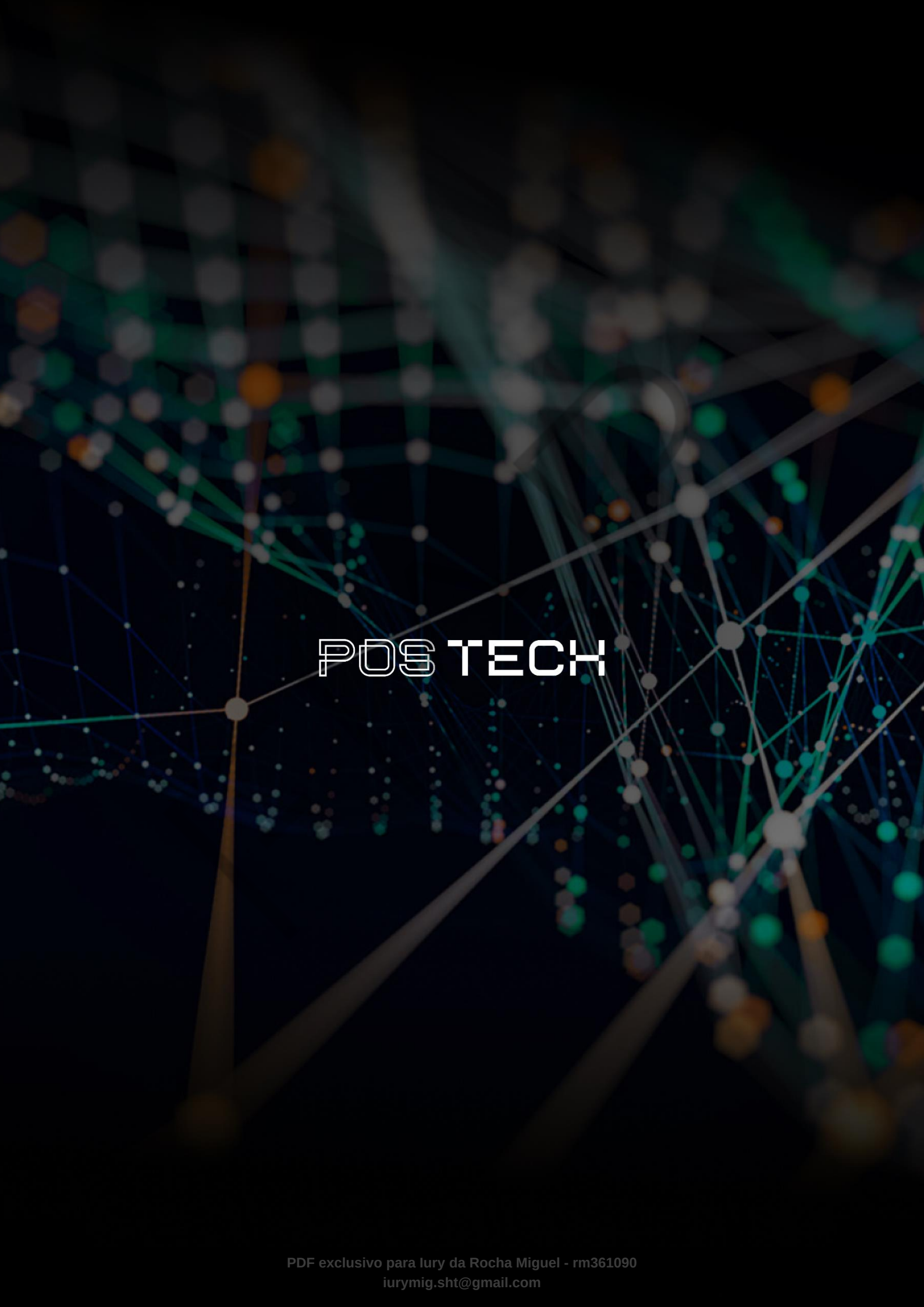
SEMMELOCK, H. **Reproducibility in Machine Learning-based Research: Overview, Barriers and Drivers**. arXiv preprint arXiv:2406.14325, 2025. Disponível em: <https://arxiv.org/abs/2406.14325>. Acesso em: 18 mar. 2026.

YIN, W. **Deep Isolation Forest for anomaly detection**. In: Advances in Neural Information Processing Systems (NeurIPS), 2020.

PALAVRAS-CHAVE

DVC. Armazenamento Remoto. GitHub.

EMENDAS



POSTECH